CHAPTER-1

INTRODUCTION TO PROJECT

1.1 Purpose of the Project

The purpose of the MenaStories. me Digital Magazine Platform is to develop a comprehensive, modern web application that allows publishers to create, manage, and distribute digital magazine content to a global audience. The project combines cutting-edge frontend technologies with a robust backend infrastructure to create an intelligent and user-friendly content management system.

Using React 18 for the frontend and Node.js with Express for the backend, the system enables seamless content creation, article management, newsletter distribution, and analytics tracking. A custom admin panel provides administrators with powerful tools to manage categories, articles, media files, and user roles.

The platform is deployed on Hostinger VPS KVM 4 infrastructure with Nginx as a reverse proxy server and PostgreSQL as the database server, ensuring enterprise-grade performance, security, and reliability.

This project aims to revolutionize digital publishing using modern web technologies, making content creation more accessible, engaging, and efficient. The platform demonstrates how modern JavaScript frameworks, cloud databases, and RESTful APIs can work together to build scalable digital publishing solutions.

The project also helps developers understand full-stack web development with real-world implementations of authentication, role-based access control, media management, SEO optimization, and production-grade deployment practices using VPS hosting.

1.2 Scope of the Project

1. **Content Management System:** The platform allows administrators and writers to create, edit, categorize, and publish articles with rich media embedding. The system supports multiple content types including text articles, video articles, flipbook magazines, and downloadable resources.

2. **Real-Time Operations**: Content updates occur in real-time with immediate feedback through WebSocket integration. Users receive instant notifications for comments, article approvals, and system events.

3. **React Frontend Application:** The frontend is developed using React 18 with Vite, providing cross-platform compatibility for all modern browsers. The application features advanced animations using Framer Motion and GSAP, creating an engaging user experience.

4. **RESTful API Communication:** The Node.js backend communicates with the frontend via RESTful APIs over HTTP/HTTPS, ensuring efficient and secure data transmission between client and server.

5. **Production-Grade Hosting:** Deployed on Hostinger VPS KVM 4 with Nginx reverse proxy, providing high availability, SSL/TLS encryption, and optimized content delivery.

6. **Scalable Database Architecture:** The project uses PostgreSQL database server hosted on the VPS with Sequelize ORM for robust data management. The database contains 80+ models with complex relationships supporting all platform features.

7. **Role-Based Access Control:** The system supports 10 different user roles with granular permissions for content management, from Master Admin with full system control to Contributors with limited article submission access.

8. **Newsletter and Communication System:** Integrated email marketing through Brevo (Sendinblue) and WhatsApp Business API for subscriber engagement, OTP verification, and campaign management.

9. **Analytics and Reporting:** Comprehensive tracking of content performance, user behavior, SEO metrics, and engagement statistics with exportable reports in PDF, CSV, and Excel formats.

10. **SEO Optimization:** Built-in support for meta tags, JSON-LD structured data, RSS feeds, dynamic sitemaps, and schema markup for maximum search engine visibility.

11. **Foundation for Future Expansion:** The modular architecture supports easy integration of AI-powered content recommendations, voice search, multi-language support, and third-party service integrations.

1.3 Objectives of the Project

1. To develop a cost-effective alternative to commercial digital publishing platforms

2. To provide a comprehensive content management system tailored for magazine-style publishing

3. To implement enterprise-grade security with JWT authentication and role-based access control

4. To create an intuitive admin interface for non-technical content managers

5. To build a scalable architecture that can handle growing content and user bases

6. To integrate modern SEO practices for maximum search engine visibility

7. To provide real-time analytics for data-driven publishing decisions

8. To support multiple content formats including articles, videos, flipbooks, and downloads

9. To enable newsletter marketing with email and WhatsApp integration

10. To implement production-grade deployment using VPS hosting with Nginx and PostgreSQL

CHAPTER-2

TECHNOLOGY / PLATFORM USED

2.1 Frontend Technologies

2.1.1 React 18

React 18 represents the latest evolution in component-based UI development, offering features like Suspense, concurrent rendering, and automatic batching. In this magazine platform, React enables the creation of dynamic, interactive user interfaces with smooth animations and responsive layouts. The declarative syntax simplifies building complex components such as article cards, navigation menus, and admin dashboards.

Key Features Used:

- Suspense for Data Fetching with loading states
- Concurrent Rendering for improved performance
- Automatic Batching for state updates
- useTransition Hook for non-blocking UI updates
- Server Components Ready architecture
- Improved Memory Management

2.1.2 Vite

Vite provides lightning-fast development with Hot Module Replacement (HMR), enabling developers to see changes instantly without full page reloads. This dramatically accelerates the development cycle and improves debugging efficiency.

Key Features Used:

- Native ESM-based development server
- Instant Hot Module Replacement
- Optimized production builds with Rollup
- CSS code splitting
- Asset optimization and hashing
- Environment variable handling

2.1.3 Framer Motion

Used for professional-grade animations and gesture recognition, Framer Motion powers the platform's page transitions, hover effects, and scroll-triggered animations, creating an engaging user experience.

Key Features Used:

- Declarative animation syntax
- Page transition animations
- Gesture support (drag, hover, tap)
- Layout animations for smooth reordering
- Scroll-triggered animations
- Exit animations with AnimatePresence

2.1.4 GSAP (GreenSock Animation Platform)

Implements advanced animations including magnetic button effects, timeline orchestrations, and hardware-accelerated transforms for smooth performance across all browsers.

Key Features Used:

- ScrollTrigger for scroll-based animations
- Timeline orchestration for complex sequences
- Magnetic cursor effects
- Text animations and reveals
- Performance-optimized transforms
- Cross-browser compatibility

2.1.5 Tailwind CSS

A utility-first CSS framework with extensive customization, providing consistent styling across the platform with a comprehensive design system.

Key Features Used:

- Utility-first approach for rapid styling
- Custom configuration with 50+ color variables
- Responsive design utilities

- Dark mode support
- Component extraction patterns
- JIT (Just-In-Time) compilation

2.1.6 Lucide Icons

Provides modern, consistent iconography throughout the application, enhancing visual hierarchy and user navigation.

Key Features Used:

- 1000+ consistent icons
- Tree-shakeable imports
- Customizable size and stroke
- React component integration
- Accessibility support

2.2 Backend Technologies

2.2.1 Node.js

The JavaScript runtime environment powers the backend server, enabling efficient handling of concurrent requests and real-time operations through its event-driven, non-blocking I/O architecture.

Key Features Used:

- Event-driven architecture for scalability
- Non-blocking I/O operations
- NPM ecosystem with extensive packages
- Native ES modules support
- Cluster mode for multi-core utilization
- Built-in debugging capabilities

2.2.2 Express.js

A minimal and flexible Node.js web application framework that provides robust routing, middleware support, and API development capabilities for the magazine platform's backend services.

Key Features Used:

- RESTful API routing
- Middleware pipeline architecture
- Error handling middleware
- Static file serving
- Request/Response helpers
- Template engine integration

2.2.3 PostgreSQL

A powerful, open-source relational database system installed directly on the Hostinger VPS, providing full-text search capabilities, advanced indexing, and reliable data storage for articles, users, and media files. PostgreSQL runs as a system service on the VPS, ensuring low latency and direct database access.

Key Features Used:

- ACID compliance for data integrity
- Full-text search for content discovery
- JSON/JSONB support for flexible data
- Advanced indexing strategies
- Connection pooling with pg-pool
- Point-in-time recovery backups
- Role-based database access control

PostgreSQL Configuration on VPS:

- Version: PostgreSQL 14+
- Max Connections: 100
- Shared Buffers: 256MB
- Work Memory: 4MB

- Maintenance Work Memory: 64MB

2.2.4 Sequelize ORM

An object-relational mapping library that simplifies database interactions through JavaScript objects, supporting associations, migrations, and data validation.

Key Features Used:

- Model definitions with validations
- Association management (1:1, 1:M, M:N)
- Migration and seeding scripts
- Query building with operators
- Hooks for lifecycle events
- Transaction support

2.2.5 JWT (JSON Web Tokens)

Provides secure, token-based authentication for user sessions, enabling stateless authorization across API endpoints.

Key Features Used:

- Stateless authentication
- Token expiration management (24h default)
- Role-based claims
- Refresh token rotation
- Secure token signing with secret key

2.3 Hosting Infrastructure

2.3.1 Hostinger VPS KVM 4

The application is hosted on Hostinger's VPS KVM 4 plan, providing dedicated resources and full root access for optimal performance and security.

VPS Specifications:

- CPU: 4 vCPU Cores
- RAM: 8 GB DDR4

- Storage: 100 GB NVMe SSD
- Bandwidth: 8 TB Monthly
- Operating System: Ubuntu 22.04 LTS
- Virtualization: KVM (Full Virtualization)
- Dedicated IP: Yes
- Root Access: Full SSH Access

Why Hostinger VPS KVM 4:

- Full virtualization ensures dedicated resources
- NVMe SSD provides fast I/O for database operations
- Sufficient RAM for Node.js, PostgreSQL, and Nginx
- Scalable to higher plans as traffic grows
- 99.9% uptime guarantee
- Data center locations worldwide
- Competitive pricing for resources provided

2.3.2 Nginx Web Server

Nginx serves as the reverse proxy server and handles all incoming HTTP/HTTPS traffic, providing load balancing, SSL termination, and static file serving capabilities.

Nginx Configuration Features:

- Reverse Proxy: Routes requests to Node.js application (port 5000)
- SSL/TLS Termination: Handles HTTPS encryption with Let's Encrypt certificates
- Static File Serving: Serves React build files and media assets
- Gzip Compression: Reduces bandwidth and improves load times
- Rate Limiting: Protects against DDoS and brute force attacks
- Security Headers: Adds security headers (X-Frame-Options, CSP, etc.)
- Caching: Browser and proxy caching for static assets
- HTTP/2 Support: Faster page loads with multiplexing

2.3.3 PostgreSQL Database Server

PostgreSQL is installed and runs directly on the VPS, providing low-latency database access and full control over database configuration and security.

Database Server Configuration:

- **Version**: PostgreSQL 14. x
- **Port**: 5432 (localhost only)
- **Authentication**: md5 password authentication
- **Max Connections**: 100
- **Shared Buffers**: 2GB (25% of RAM)
- **Effective Cache Size**: 6GB (75% of RAM)
- **Work Memory**: 16MB
- **Maintenance Work Memory**: 512MB

Backup Strategy:

- Daily automated backups using pg_dump
- Weekly full backups retained for 30 days
- Backup files stored in separate directory
- Optional offsite backup to cloud storage

2.3.4 Process Management with PM2

PM2 is used to manage the Node.js application, ensuring automatic restarts, load balancing, and monitoring.

PM2 Features Used:

- Cluster mode for utilizing multiple CPU cores
- Automatic restart on crashes
- Log management and rotation
- Memory limit monitoring
- Zero-downtime deployments

2.4 Security Components

2.4.1 Helmet. js

Secures Express apps by setting various HTTP headers, protecting against common web vulnerabilities including XSS, clickjacking, and MIME-type sniffing.

2.4.2 XSS-Clean

Middleware for sanitizing user input to prevent cross-site scripting attacks by removing malicious scripts from request data.

2.4.3 Express-Validator

Comprehensive validation library for request data validation and sanitization, ensuring only properly formatted data reaches the application logic.

2.4.4 Rate Limiting

Protects API endpoints from abuse by limiting the number of requests from a single IP address within a specified time window.

2.4.5 CORS Configuration

Manages cross-origin resource sharing policies for secure API access from authorized domains while blocking unauthorized origins.

2.4.6 UFW Firewall

Uncomplicated Firewall configured on the VPS to allow only necessary ports:

- Port 22: SSH (restricted to specific IPs recommended)
- Port 80: HTTP (redirects to HTTPS)
- Port 443: HTTPS
- Port 5432: PostgreSQL (localhost only, blocked externally)

2.5 External Services

2.5.1 Cloudinary

Cloud-based media management platform for storing, optimizing, and delivering images and videos across the magazine website with automatic format conversion and responsive image delivery.

Capabilities:

- Image and video upload

- Automatic format optimization (WebP, AVIF)
- Responsive image transformations
- CDN delivery worldwide
- AI-powered tagging

2.5.2 Brevo (Sendinblue)

Email service provider for newsletter management, subscription confirmations, and automated email campaigns with detailed analytics and deliverability tracking.

Capabilities:

- Transactional email delivery
- Newsletter campaigns
- Email template management
- Delivery analytics
- SMTP relay service

2.5.3 WhatsApp Business API

Messaging platform integration for OTP verification, subscriber notifications, and marketing campaigns reaching users directly on their mobile devices.

Capabilities:

- OTP message delivery
- Notification broadcasts
- Template message support
- Delivery status tracking
- Phone number verification

2.5.4 Let's Encrypt

Free, automated SSL/TLS certificate authority providing HTTPS encryption for the website. Certificates are automatically renewed using Certbot.

Configuration:

- Auto-renewal every 60 days

- Wildcard certificate support
- OCSP stapling enabled
- Strong cipher configuration

CHAPTER – 3

EXISTING SYSTEM

3.1 Traditional Digital Publishing Platforms

Traditional digital magazine platforms are commonly provided by large publishing companies such as Issuu, Flipboard, Medium, and proprietary CMS solutions like WordPress with premium themes. These systems offer advanced features like content management, reader analytics, subscription management, and multi-platform distribution. While technologically sophisticated, these systems come with a range of limitations.

3.2 Limitations of Existing Systems

3.2.1 High Cost

Most commercially available digital publishing platforms require expensive monthly subscriptions ranging from $50 to $500+ per month, enterprise licenses costing thousands annually, or significant investment in custom development. Additionally, many platforms charge based on traffic, storage, or the number of publications, making them inaccessible for startups, independent publishers, or educational institutions.

Cost Breakdown of Popular Platforms:

- WordPress Premium with Publishing Theme: $200-500/year + hosting ($20-100/month)
- Issuu Premium: $228-468/year
- Medium Partner Program: Revenue sharing model (30% platform cut)
- Custom CMS Development: $10,000-50,000+ initial investment
- Managed WordPress Hosting: $30-200/month

3.2.2 Limited Customization

These systems are often locked into specific templates, themes, or publishing workflows, limiting how much publishers can modify the reading experience, design aesthetics, or content organization structure. Customization typically requires expensive developer intervention or premium plan upgrades.

3.2.3 Vendor Lock-in

Publishers become dependent on specific platforms, making it difficult to migrate content, subscriber lists, and analytics data to alternative solutions. Data portability is often limited or requires technical expertise to execute.

3.2.4 Technical Dependency and Complexity

Traditional systems require either professional developers, expensive consultants, or advanced technical knowledge. This discourages many publishers from implementing custom automation or unique features that could differentiate their publication.

3.2.5 Generic Features

One-size-fits-all solutions don't account for the specific needs of magazine-style publishing, such as power lists, flipbook magazines, event coverage, or specialized content categorization.

3.2.6 Limited Role System

Most platforms offer only basic user types (admin, editor, author, subscriber) without the granular permission control needed for complex editorial workflows involving fact-checkers, section editors, and social media managers.

3.2.7 Third-Party Analytics Dependency

Analytics typically require external integrations with Google Analytics or similar services, fragmenting data and requiring multiple dashboards to understand complete publication performance.

3.2.8 Shared Hosting Limitations

Many affordable solutions run on shared hosting, leading to:

- Slow page load times during traffic spikes
- Limited server configuration options
- Security vulnerabilities from neighboring sites
- No root access for optimization
- Resource throttling during peak usage

3.2.9 Lack of Open-Source Accessibility

Traditional systems do not prioritize open-source accessibility, which limits learning opportunities and experimentation for developers who wish to build custom publishing solutions or integrate with emerging technologies.

3.3 Market Analysis

The digital publishing market is dominated by a few major players, leaving independent publishers with limited affordable options. The total addressable market for digital publishing solutions exceeds $15 billion globally, yet affordable, customizable solutions with dedicated hosting remain scarce.

CHAPTER – 4

PROPOSED SYSTEM

4.1 MenaStories.me - Modern Digital Magazine Platform

The proposed system addresses the limitations of traditional digital publishing by offering a **cost-effective, flexible, and developer-friendly solution** using modern web technologies. This digital magazine platform is built using open-source technologies including React 18, Node.js, Express.js, and PostgreSQL, deployed on dedicated VPS infrastructure with Nginx.

4.2 Core Features of Proposed System

4.2.1 Comprehensive Content Management

The admin panel provides complete control over the content lifecycle:

- Rich text article editor with media embedding
- Multiple content types: articles, videos, flipbooks, downloads
- Draft, review, and publish workflow
- Version history and revision tracking
- Bulk content operations
- Scheduled publishing

4.2.2 Advanced Categorization System

- 8 main categories with 1666+ subcategories
- Dynamic category design templates (3 layout options)
- Hierarchical tag system
- Special features and trending content sections
- Cross-category article linking

4.2.3 Multi-Role Editorial Workflow

- 10 distinct user roles with specific permissions
- Content review and approval pipeline
- Fact-checking workflow integration

- Collaborative editing capabilities
- Activity logging and audit trails

4.2.4 Integrated Newsletter System

- Email subscription with double opt-in
- WhatsApp integration for notifications
- Campaign management and scheduling
- Template library for consistent branding
- Detailed delivery and engagement analytics

4.2.5 Event and Exhibition Management

- Event creation and management
- Registration system with capacity limits
- Event updates and notifications
- Exhibition galleries
- Event analytics and reporting

4.2.6 Flipbook Magazine Feature

- PDF to flipbook conversion
- Page-by-page navigation
- Thumbnail generation
- Reading analytics
- Download options

4.2.7 Power Lists and Rankings

- Dynamic list creation (Top 30, Top 50, etc.)
- Entry management with rankings
- Profile pages for list entries
- Historical archive of past lists
- SEO-optimized list pages

4.2.8 Publisher Portal

- External publisher registration
- Content submission workflow
- Publisher dashboard with analytics
- Activity logging
- Approval/rejection management

4.3 Infrastructure Advantages

4.3.1 Dedicated VPS Resources

Unlike shared hosting solutions, Hostinger VPS KVM 4 provides:

- **Guaranteed Resources**: 4 vCPU, 8GB RAM dedicated to the application
- **NVMe SSD Storage**: Fast I/O for database and file operations
- **Full Root Access**: Complete control over server configuration
- **Isolation**: No interference from other users or applications
- **Scalability**: Easy upgrade path to higher VPS tiers

4.3.2 Nginx Performance Benefits

Using Nginx as reverse proxy provides:

- **High Concurrency**: Handle thousands of simultaneous connections
- **Low Memory Footprint**: Efficient resource utilization
- **Static File Serving**: Offload static assets from Node.js
- **SSL Termination**: Efficient HTTPS handling
- **Load Balancing**: Distribute traffic across Node.js instances
- **Caching**: Reduce backend load with response caching

4.3.3 Local PostgreSQL Advantages

Running PostgreSQL on the same VPS provides:

- **Low Latency**: No network overhead for database queries
- **Full Control**: Custom configuration for workload optimization

- **Security**: Database accessible only from localhost
- **Cost Savings**: No separate database hosting fees
- **Backup Control**: Custom backup schedules and retention

4.4 Technical Innovations

4.4.1 Revolutionary Interface Design

The platform reimagines how users interact with digital content through:

- Glassmorphism UI with backdrop blur effects
- Magnetic cursor interactions
- Gradient text masking effects
- Micro-animations throughout the interface
- 3D visual depth with layered shadows
- Dynamic floating particles and backgrounds

4.4.2 Performance Optimization

- Sub-3 second initial page load
- Lazy loading for images and components
- Code splitting for reduced bundle sizes
- CDN delivery for static assets via Cloudinary
- Database query optimization with indexing
- Nginx caching for frequently accessed content
- Gzip/Brotli compression

4.4.3 SEO Excellence

- Automatic sitemap generation
- JSON-LD structured data for all content types
- RSS feed generation
- Meta tag management
- Schema markup for rich snippets

CHAPTER – 5
SYSTEM DEVELOPMENT PROCESS

5.1 System Analysis

5.1.1 Data Analysis

Before developing the MenaStories.me platform, an in-depth analysis of existing digital publishing systems and content management solutions was conducted. The analysis revealed:

Content Requirements:

- Article storage with rich text formatting (HTML/Markdown)
- Media file management (images up to 10MB, videos up to 100MB)
- Category hierarchies up to 3 levels deep
- Tag-based content organization with M: N relationships
- Version control for content revisions

User Requirements:

- Multiple user roles with different permissions (10 roles identified)
- Authentication and session management (JWT with 24h expiry)
- Profile management capabilities
- Activity tracking and audit logs

Infrastructure Requirements:

- High availability (99.9% uptime target)
- Sub-200ms API response times
- Support for 1000+ concurrent users
- 100GB+ storage capacity
- Automated backups

Operational Requirements:

- 24/7 availability for public content
- Admin access with role-based restrictions
- Backup and recovery procedures (daily automated backups)
- Performance monitoring and alerting

5.1.2 Operational Analysis

Operational analysis evaluates how the system integrates into publishing workflows:

Daily Operations:

- Content creation by writers and contributors
- Editorial review and approval
- Media upload and organization
- Newsletter preparation and sending
- Analytics review and reporting

Weekly Operations:

- Category and tag maintenance
- User permission reviews
- Performance monitoring via PM2 and Nginx logs
- Backup verification

Monthly Operations:

- Comprehensive analytics review
- SEO performance assessment
- Security audit and system updates
- VPS resource utilization review
- Feature enhancement planning

Server Maintenance Tasks:

- Daily: Automated PostgreSQL backups
- Weekly: Log rotation and cleanup
- Monthly: System updates (apt upgrade)
- Quarterly: SSL certificate renewal (automated)

5.1.3 Technical Analysis

From a technical standpoint, the system is highly feasible:

Technology Availability:

- All frameworks are open-source and well-documented
- Large developer communities for support
- Extensive package ecosystems (npm with 2M+ packages)
- VPS hosting available at multiple price points
- PostgreSQL and Nginx widely supported

Integration Capabilities:

- RESTful APIs for third-party integration

- Webhook support for automation
- OAuth integration possibilities
- Standard data formats (JSON, XML)
- SMTP/API email integration

Maintenance Requirements:

- Regular dependency updates (monthly)
- Security patch management (as released)
- Database optimization (vacuum, analyze)
- Nginx and PostgreSQL configuration tuning
- PM2 process monitoring

5.1.4 Economic Analysis

The economic evaluation demonstrates financial viability:

Cost Category	Commercial Solution	MenaStories.me	Annual Savings
Licensing	$2,000+	$0	$2,000+
VPS Hosting (KVM 4)	N/A	$180	-
Managed Hosting	$1,200+	$0	$1,200+
Database (External)	$300+	$0	$300+
Development	$5,000	$0 (self)	$5,000
Maintenance	$1,200	$300	$900
Training	$500	$100	$400
Total	$10,200+	$580	$9,620+

3-Year Total Cost of Ownership:

- Commercial Solution: $30,000+
- MenaStories.me on VPS: $1,740
- Total Savings: $28,000+

5.2 Feasibility Study

5.2.1 Economic Feasibility

The proposed system is significantly more affordable than existing market solutions:

- Uses entirely open-source technologies ($0 licensing)
- VPS hosting at $15-20/month with dedicated resources
- PostgreSQL included on VPS (no separate database cost)

- Nginx free and open-source
- Self-hosted option for complete cost control
- No recurring subscription fees

Return on Investment (ROI):

- Break-even within 1 month compared to commercial alternatives
- Long-term savings of $9,000+ annually
- Scalable costs that grow predictably with the publication

5.2.2 Technical Feasibility

Using proven technologies ensures technical viability:

- React 18: Stable release with long-term support (April 2022+)
- Node.js 18 LTS: Supported until April 2025
- PostgreSQL 14: Enterprise-grade with 5+ year support
- Nginx: Industry standard with 20+ years of development
- Hostinger VPS: Reliable infrastructure with 99.9% SLA

Technical Requirements Met:

- All functional requirements achievable
- Performance targets realistic with VPS resources
- Security standards implementable
- Scalability patterns established
- Disaster recovery possible

5.2.3 Operational Feasibility

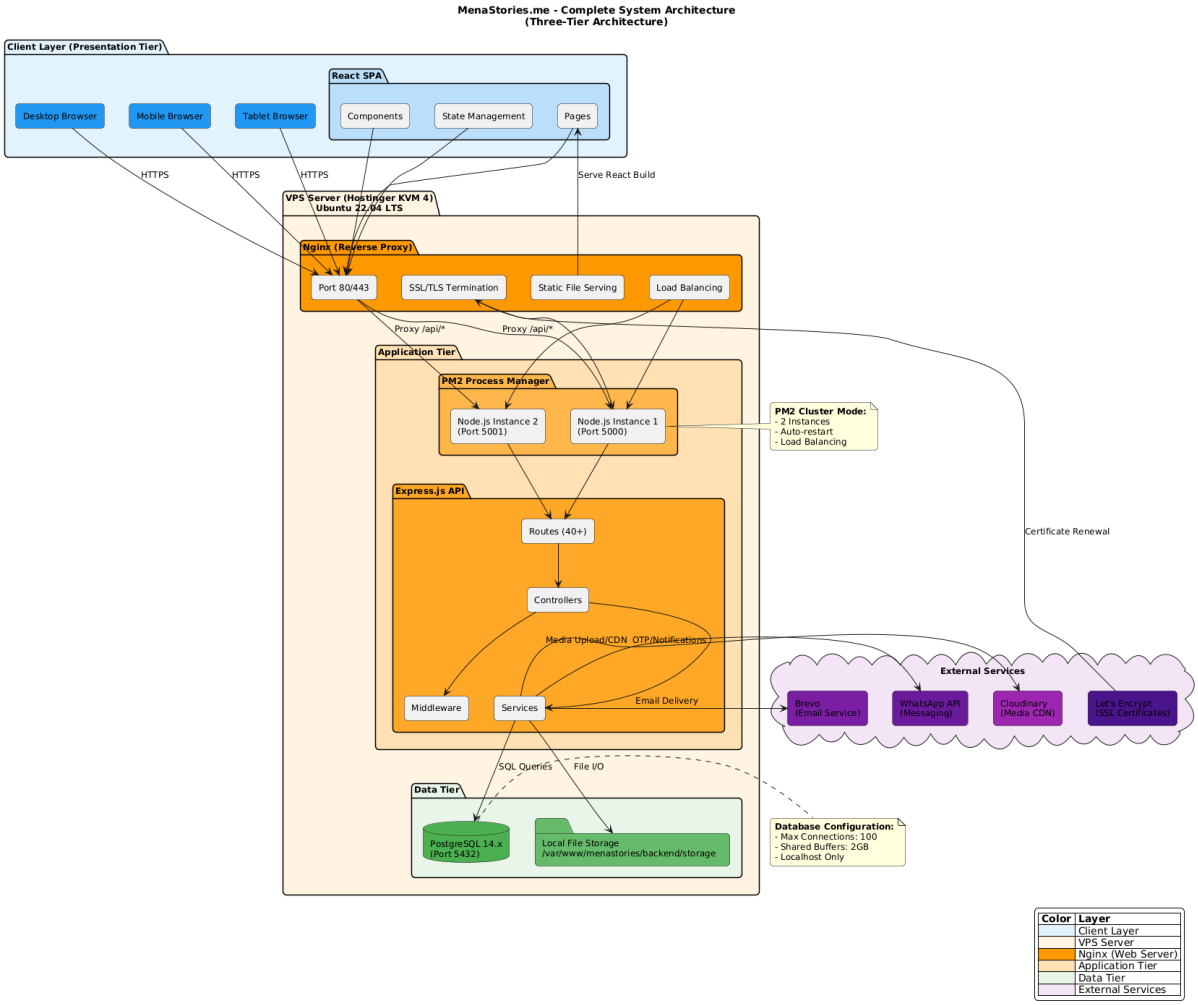
The system integrates smoothly into publishing workflows:

- Intuitive interface reduces training time (2-4 hours for basic use)
- Familiar patterns for content management
- Mobile-responsive for on-the-go access
- Real-time collaboration features
- SSH access for technical operations

CHAPTER-6

SYSTEM DESIGN

6.1 System Architecture



The MenaStories.me platform follows a modern three-tier architecture deployed on Hostinger VPS:

6.1.1 Presentation Tier (Client Layer)

The frontend application built with React 18:

- Single Page Application (SPA) architecture
- Component-based UI structure
- State management with Context API
- Client-side routing with React Router

- Responsive design for all devices
- Served as static files by Nginx

6.1.2 Application Tier (Business Logic Layer)

The Node.js/Express backend server managed by PM2:

- RESTful API endpoints (40+ route files)
- Authentication and authorization middleware
- Business logic processing in controllers
- Data validation and sanitization
- External service integration
- WebSocket for real-time features

6.1.3 Data Tier (Database Layer)

PostgreSQL database with supporting services:

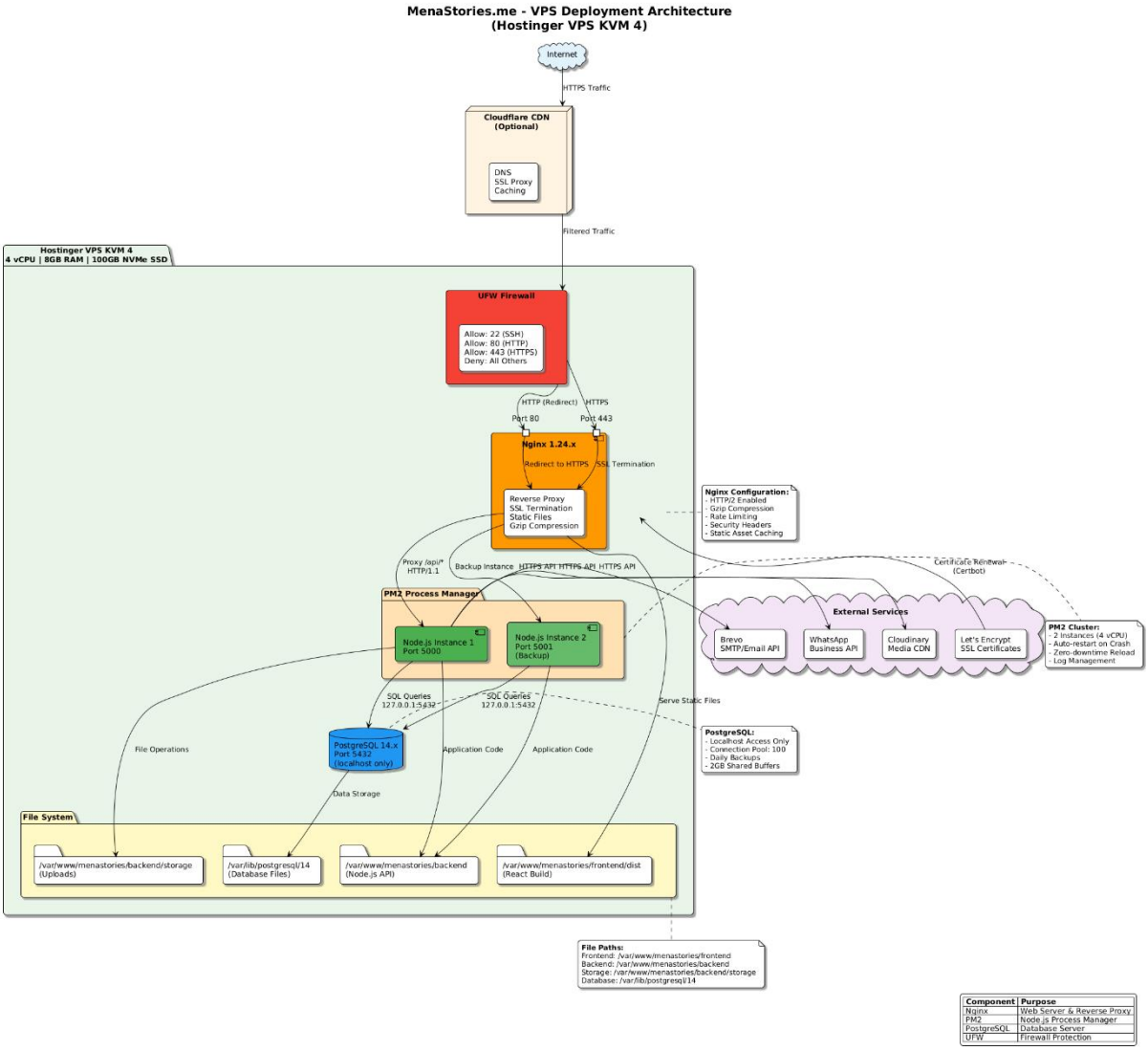
- Relational data storage (80+ tables)
- Full-text search capabilities
- Local file storage for uploads
- Cloudinary for media CDN

6.2 Deployment Architecture

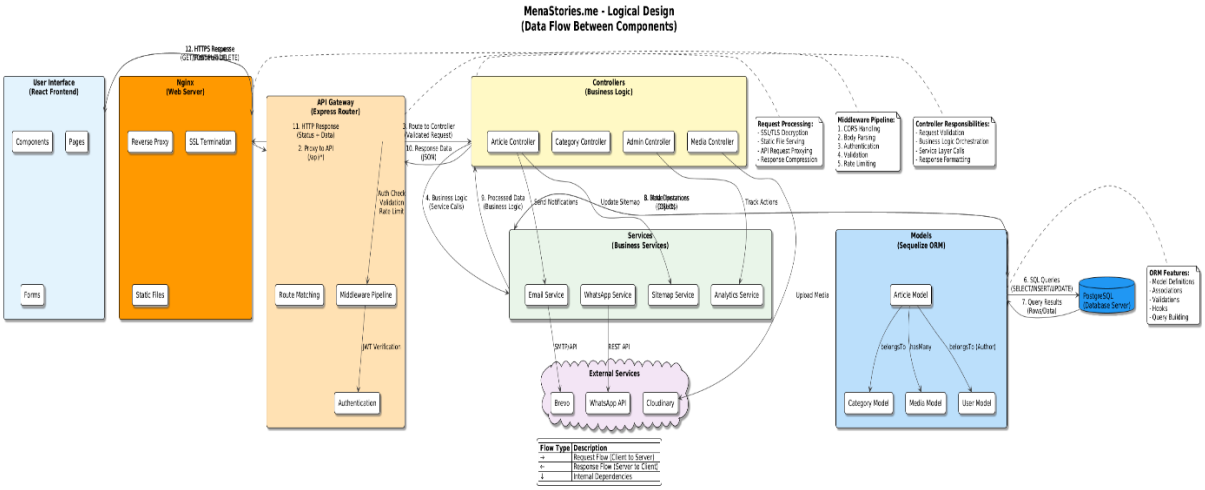
6.2.1 Service Configuration

Systemd Services Running:

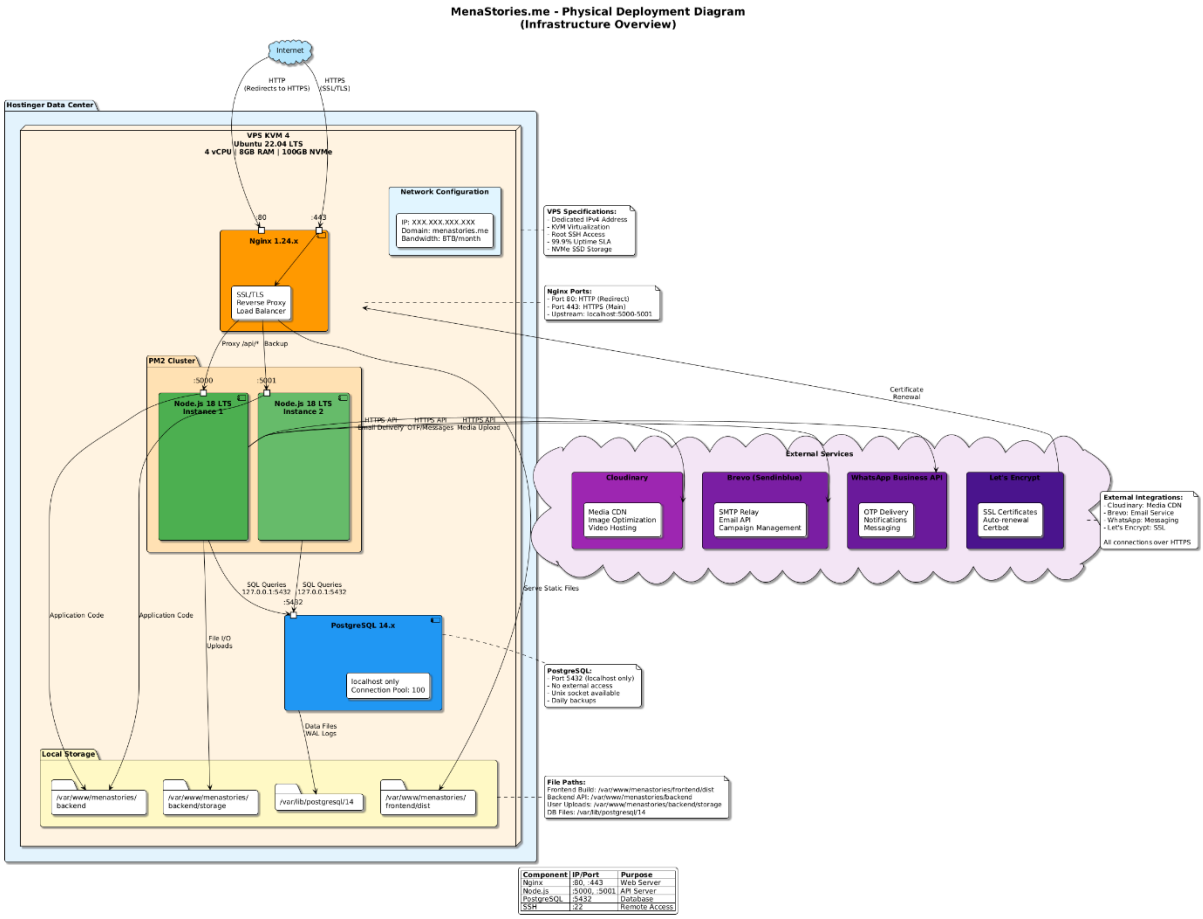
- nginx. service: Web server and reverse proxy
- postgresql.service: Database server
- pm2-root.service: Node.js process manager



6.3 Logical Design



6.4 Physical Design



The physical design describes the deployment architecture:

6.4.1 Production Environment

VPS Server Specifications (Hostinger KVM 4):

- CPU: 4 vCPU cores
- RAM: 8 GB DDR4
- Storage: 100 GB NVMe SSD
- OS: Ubuntu 22.04 LTS
- Location: Configurable (US, EU, Asia)

Software Stack:

- Nginx 1.24.x
- Node.js 18.x LTS
- PM2 5.x
- PostgreSQL 14.x

- Certbot (Let's Encrypt)
- UFW Firewall

Resource Allocation:

- Node.js (PM2): ~2GB RAM
- PostgreSQL: ~2GB RAM (shared_buffers)
- Nginx: ~256MB RAM
- OS and buffers: ~3. 7GB RAM
- Available for growth: ~2GB

6.4.2 Development Environment

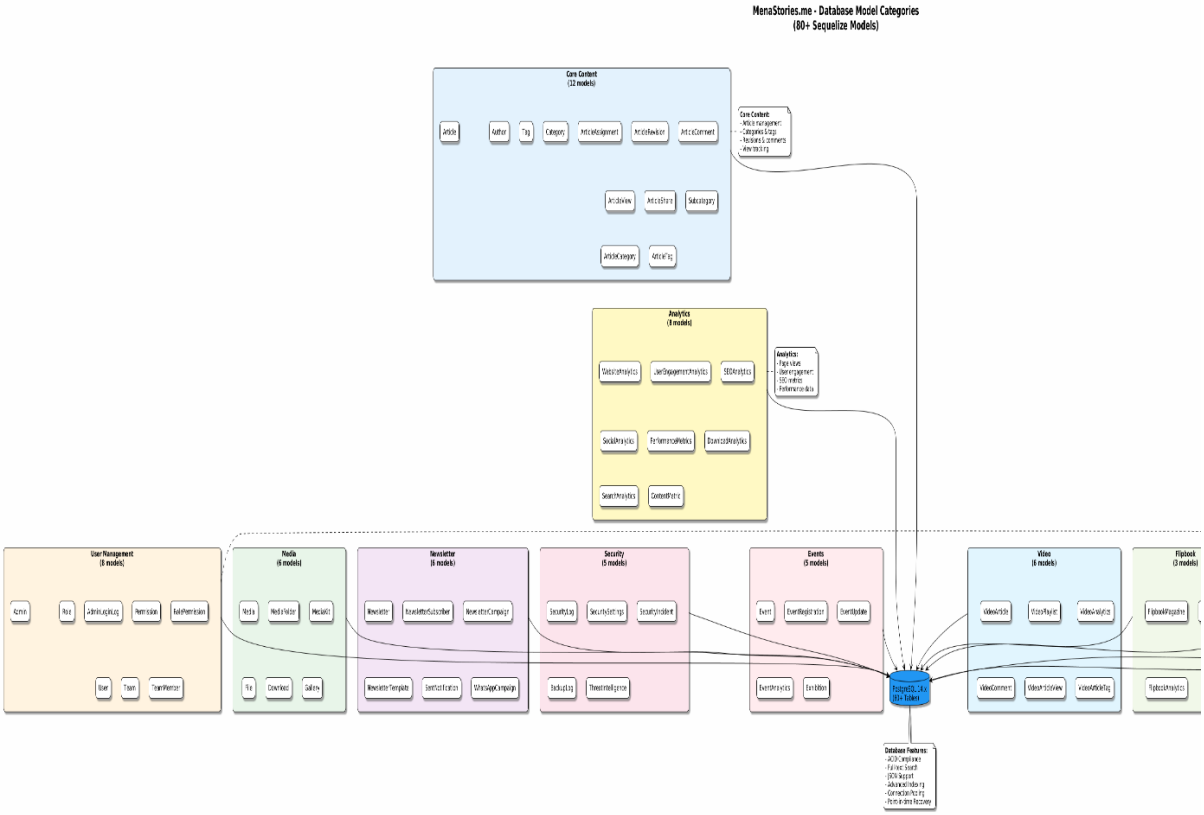
- Local Development: Vite dev server + Node.js
- Database: SQLite (local) or PostgreSQL (Docker/local)
- Hot Reloading: Enabled for rapid development
- Debug Tools: React DevTools, Node. js debugger
- Version Control: Git with GitHub

CHAPTER-7

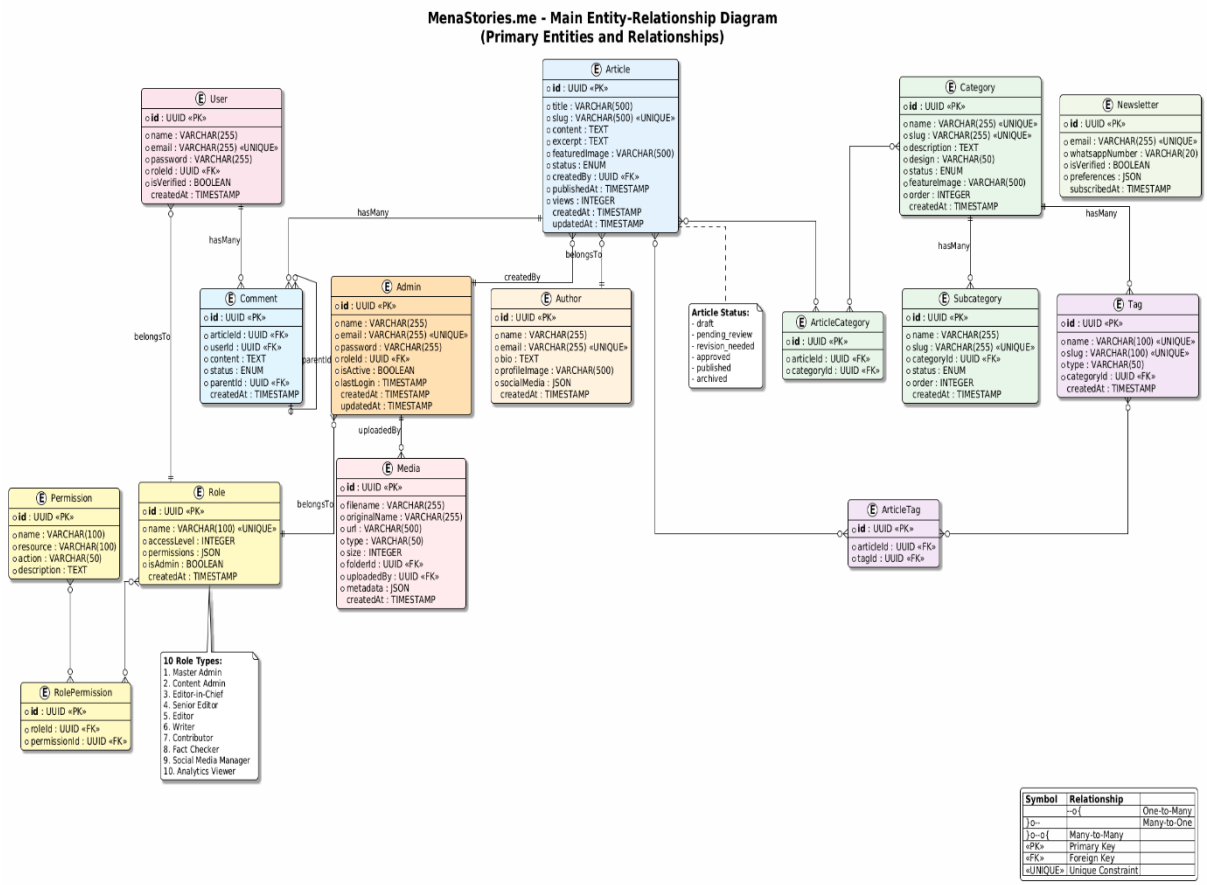
DATABASE DESIGN

7.1 Database Overview

The MenaStories. me platform uses PostgreSQL 14.x as its primary database, running directly on the Hostinger VPS. The database is managed through Sequelize ORM and contains 80+ models organized into logical groups.



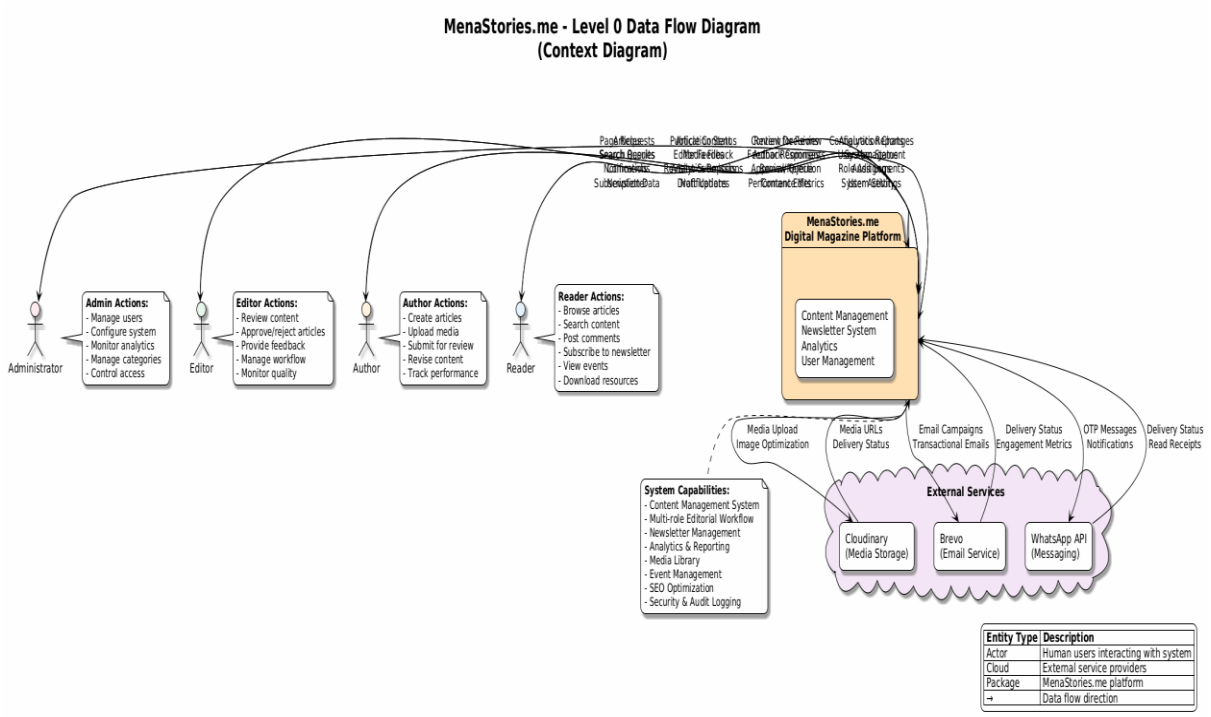
7.2 Entity-Relationship Diagram


MenaStories.me - Main Entity-Relationship Diagram
(Primary Entities and Relationships)
User
id : UUID «PK»
name : VARCHAR(255)
email : VARCHAR(255) «UNIQUE»
password : VARCHAR(255)
roleId : UUID «FK»
isVerified : BOOLEAN
createdAt : TIMESTAMP
Article
id : UUID «PK»
title : VARCHAR(500)
slug : VARCHAR(500) «UNIQUE»
content : TEXT
excerpt : TEXT
featuredImage : VARCHAR(500)
status : ENUM
createdBy : UUID «FK»
publishedAt : TIMESTAMP
views : INTEGER
createdAt : TIMESTAMP
updatedAt : TIMESTAMP
Category
id : UUID «PK»
name : VARCHAR(255) «UNIQUE»
slug : VARCHAR(255) «UNIQUE»
description : TEXT
design : VARCHAR(50)
status : ENUM
featureImage : VARCHAR(500)
order : INTEGER
createdAt : TIMESTAMP
Newsletter
id : UUID «PK»
email : VARCHAR(255) «UNIQUE»
whatsappNumber : VARCHAR(20)
isVerified : BOOLEAN
preferences : JSON
subscribedAt : TIMESTAMP
hasMany
hasMany
hasMany
hasMany
belongsTo
Comment
id : UUID «PK»
articleId : UUID «FK»
userId : UUID «FK»
content : TEXT
status : ENUM
parentId : UUID «FK»
createdAt : TIMESTAMP
parentId
Admin
id : UUID «PK»
name : VARCHAR(255)
email : VARCHAR(255) «UNIQUE»
password : VARCHAR(255)
roleId : UUID «FK»
isActive : BOOLEAN
lastLogin : TIMESTAMP
createdAt : TIMESTAMP
updatedAt : TIMESTAMP
createdBy
Author
id : UUID «PK»
name : VARCHAR(255)
email : VARCHAR(255) «UNIQUE»
bio : TEXT
profileImage : VARCHAR(500)
socialMedia : JSON
createdAt : TIMESTAMP
Article Status:
- draft
- pending_review
- revision_needed
- approved
- published
- archived
ArticleCategory
id : UUID «PK»
articleId : UUID «FK»
categoryId : UUID «FK»
Subcategory
id : UUID «PK»
name : VARCHAR(255)
slug : VARCHAR(255) «UNIQUE»
categoryId : UUID «FK»
status : ENUM
order : INTEGER
createdAt : TIMESTAMP
Tag
id : UUID «PK»
name : VARCHAR(100) «UNIQUE»
slug : VARCHAR(100) «UNIQUE»
type : VARCHAR(50)
categoryId : UUID «FK»
createdAt : TIMESTAMP
belongsTo
uploadedBy
Media
id : UUID «PK»
filename : VARCHAR(255)
originalName : VARCHAR(255)
url : VARCHAR(500)
type : VARCHAR(50)
size : INTEGER
folderId : UUID «FK»
uploadedBy : UUID «FK»
metadata : JSON
createdAt : TIMESTAMP
Permission
id : UUID «PK»
name : VARCHAR(100)
resource : VARCHAR(100)
action : VARCHAR(50)
description : TEXT
Role
id : UUID «PK»
name : VARCHAR(100) «UNIQUE»
accessLevel : INTEGER
permissions : JSON
isAdmin : BOOLEAN
createdAt : TIMESTAMP
belongsTo
ArticleTag
id : UUID «PK»
articleId : UUID «FK»
tagId : UUID «FK»
RolePermission
id : UUID «PK»
roleId : UUID «FK»
permissionId : UUID «FK»
10 Role Types:
1. Master Admin
2. Content Admin
3. Editor-in-Chief
4. Senior Editor
5. Editor
6. Writer
7. Contributor
8. Fact Checker
9. Social Media Manager
10. Analytics Viewer
Symbol | Relationship
--o{ | One-to-Many
}o-- | Many-to-One
}o--o{ | Many-to-Many
«PK» | Primary Key
«FK» | Foreign Key
«UNIQUE» | Unique Constraint

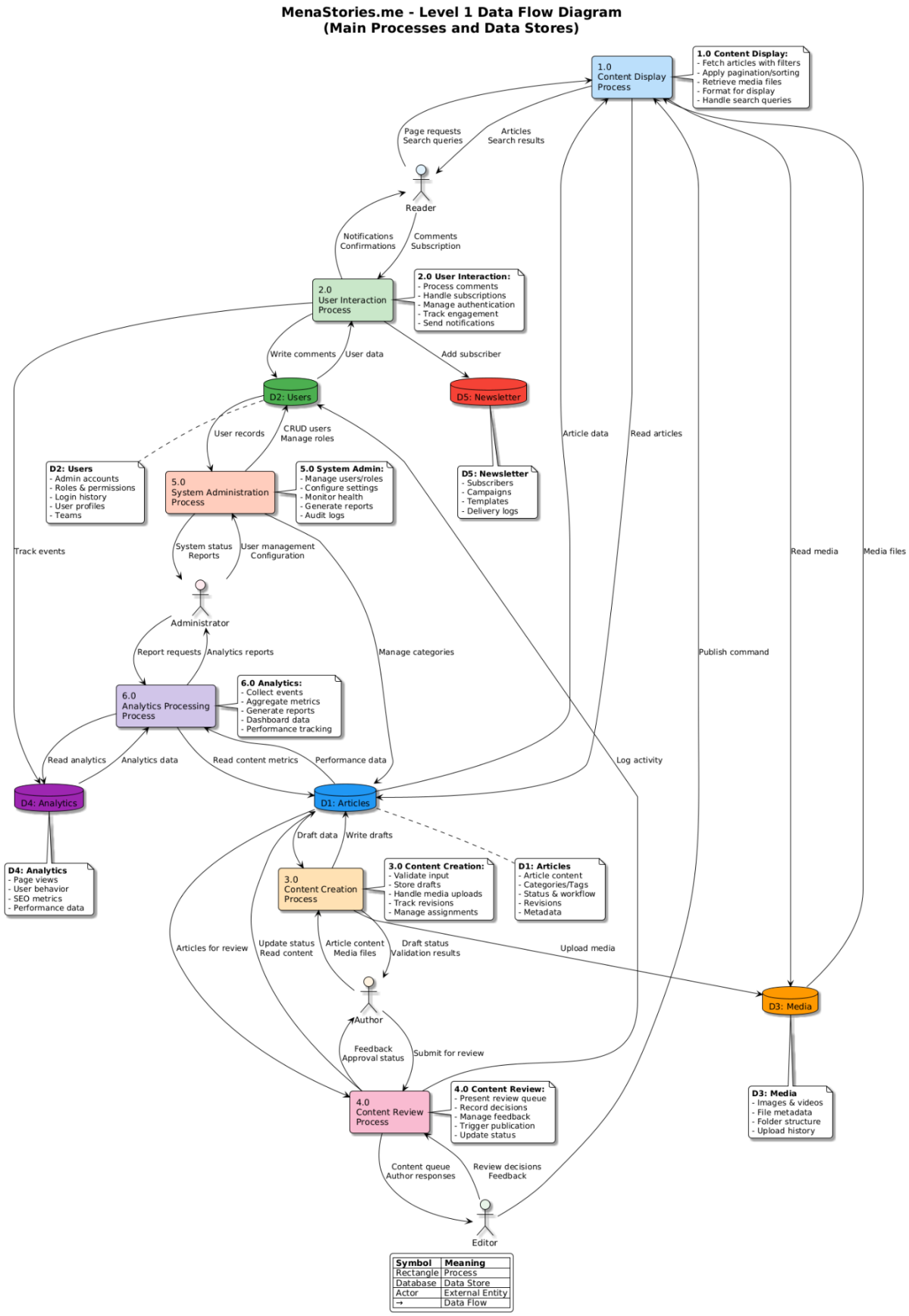
CHAPTER – 8

DATA FLOW DIAGRAMS

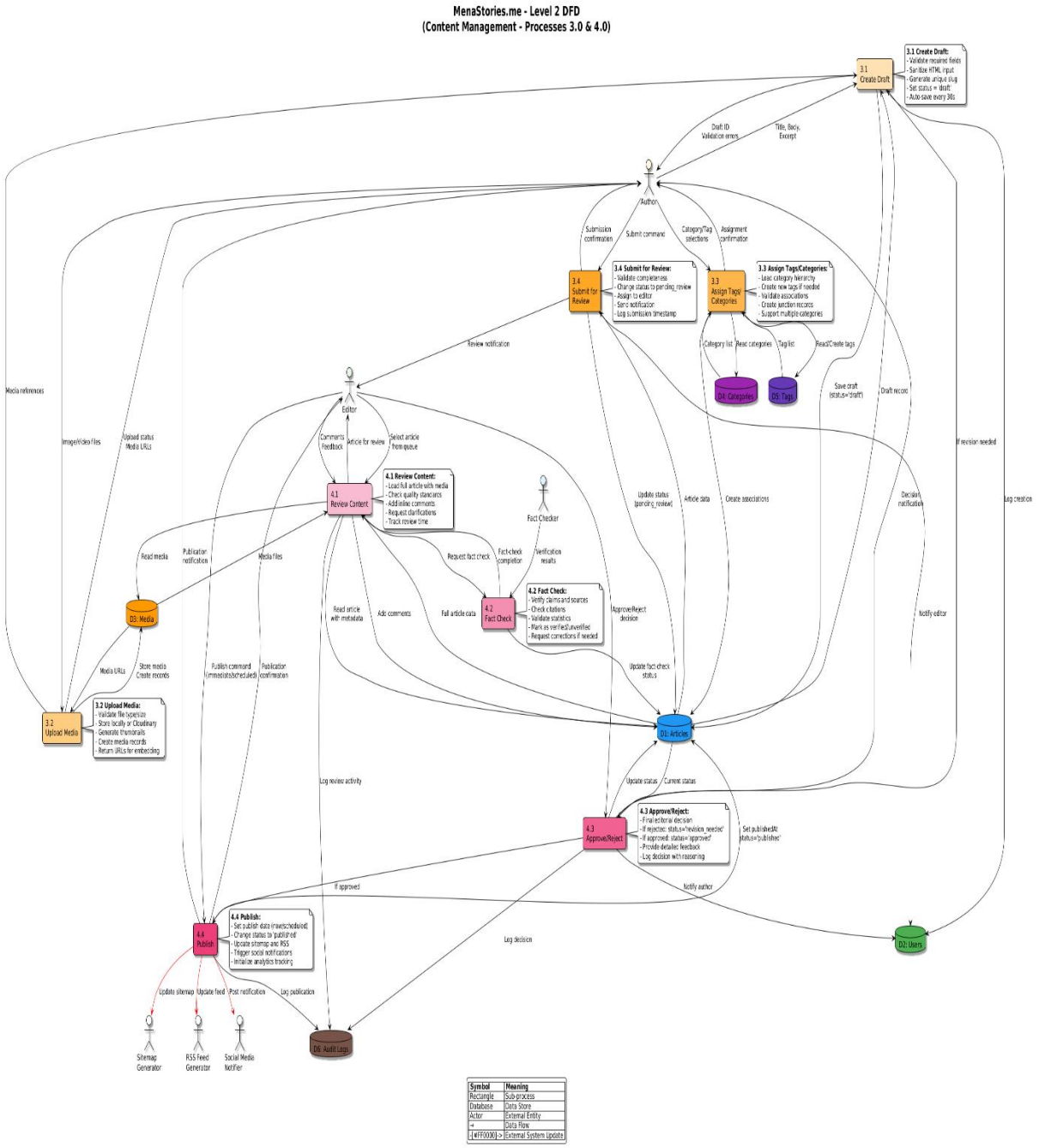
8.1 Level 0 DFD (Context Diagram)



8.2 Level 1 DFD



8.3 Level 2 DFD - Content Management


MenaStories.me - Level 2 DFD
(Content Management - Processes 3.0 & 4.0)
3.1 Create Draft
3.1 Create Draft:
- Validate required fields
- Sanitize HTML input
- Generate unique slug
- Set status = 'draft'
- Auto-save every 30s
Draft ID
Validation errors
Title, Body, Excerpt
Author
Submission confirmation
Submit command
Category/Tag selections
Assignment confirmation
3.4 Submit for Review
3.4 Submit for Review:
- Validate completeness
- Change status to pending_review
- Assign to editor
- Send notification
- Log submission timestamp
3.3 Assign Tags/Categories
3.3 Assign Tags/Categories:
- Load category hierarchy
- Create new tags if needed
- Validate associations
- Create junction records
- Support multiple categories
Review notification
Category list
Read categories
Tag list
Read/Create tags
D4: Categories
D5: Tags
Save draft (status='draft')
Draft record
Media references
Image/Video files
Upload status
Media URLs
Editor
Comments
Feedback
Article for review
Select article from queue
4.1 Review Content
4.1 Review Content:
- Load full article with media
- Check quality standards
- Add inline comments
- Request clarifications
- Track review time
Fact Checker
Update status (pending_review)
Article data
Create associations
If revision needed
Decision notification
Log creation
Read media
Publication notification
Media files
Request fact check
Fact-check completion
Verification results
4.2 Fact Check
4.2 Fact Check:
- Verify claims and sources
- Check citations
- Validate statistics
- Mark as verified/unverified
- Request corrections if needed
Approve/Reject decision
Notify editor
D3: Media
Read article with metadata
Add comments
Full article data
Update fact-check status
Media URLs
Store media
Create records
Publish command (immediate/scheduled)
Publication confirmation
3.2 Upload Media
3.2 Upload Media:
- Validate file type/size
- Store locally or Cloudinary
- Generate thumbnails
- Create media records
- Return URLs for embedding
D1: Articles
Log review activity
Update status
Current status
4.3 Approve/Reject
4.3 Approve/Reject:
- Final editorial decision
- If rejected: status='revision_needed'
- If approved: status='approved'
- Provide detailed feedback
- Log decision with reasoning
Set publishedAt; status='published'
Notify author
If approved
4.4 Publish
4.4 Publish:
- Set publish date (now/scheduled)
- Change status to 'published'
- Update sitemap and RSS
- Trigger social notifications
- Initialize analytics tracking
D2: Users
Log decision
Update sitemap
Update feed
Post notification
Log publication
Sitemap Generator
RSS Feed Generator
Social Media Notifier
D6: Audit Logs
Symbol | Meaning
Rectangle | Sub-process
Database | Data Store
Actor | External Entity
→ | Data Flow
-[#FF0000]-> | External System Update

CHAPTER – 9

SOFTWARE / HARDWARE REQUIREMENTS

13.1 Development Environment Requirements

Hardware Requirements (Development):

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5 GHz+
RAM	8 GB	16 GB
Storage	50 GB SSD	100 GB SSD
Display	1366 x 768	1920 x 1080
Internet	10 Mbps	50+ Mbps

Software Requirements (Development):

Software	**Version**	**Purpose**
Node.js	18.x LTS	JavaScript runtime
npm	9.x+	Package manager
Git	2.30+	Version control
VS Code	Latest	Code editor (recommended)
PostgreSQL	14.x	Database (local)
Chrome/Firefox	Latest	Browser testing
Postman	Latest	API testing

13.2 Production Environment Requirements

Hardware Requirements (VPS - Hostinger KVM 4):

Component	**Specification**
CPU	4 vCPU Cores
RAM	8 GB DDR4
Storage	100 GB NVMe SSD
Bandwidth	8 TB/month
Network	1 Gbps

IP Address	1 Dedicated IPv4

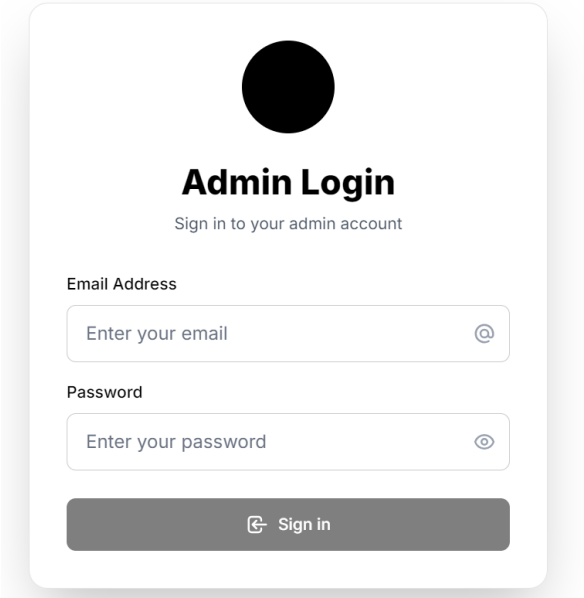
Software Requirements (Production):

Software	Version	Purpose
Ubuntu	22.04 LTS	Operating System
Nginx	1.24.x	Web server / Reverse proxy
Node.js	18.x LTS	Application runtime
PM2	5.x	Process manager
PostgreSQL	14.x	Database server
Certbot	Latest	SSL certificate management
UFW	Built-in	Firewall
Git	2.34+	Deployment

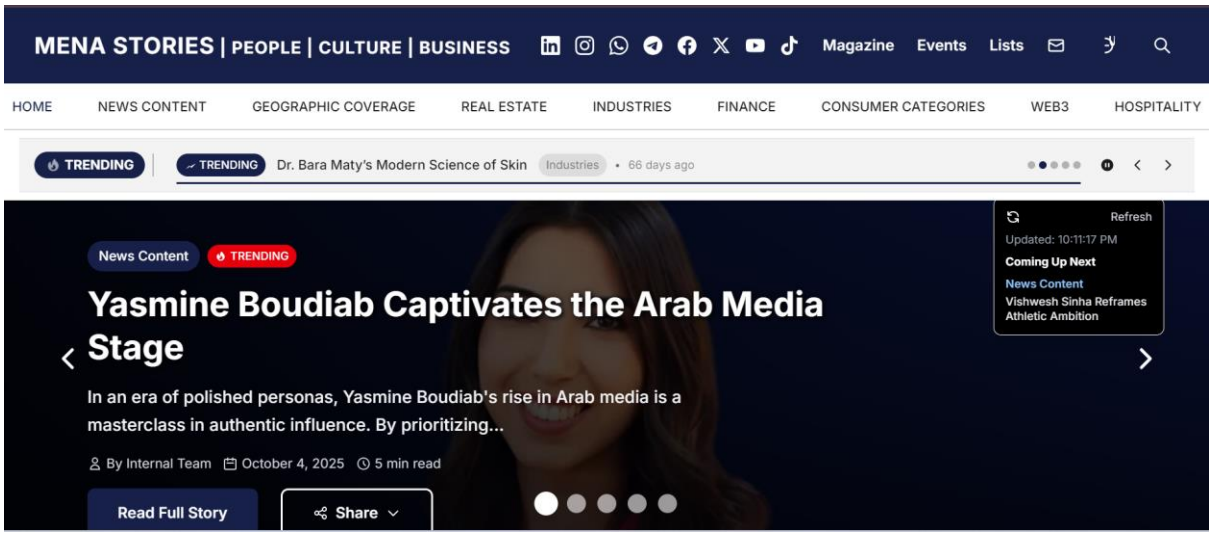
CHAPTER – 10

SNAPSHOTS

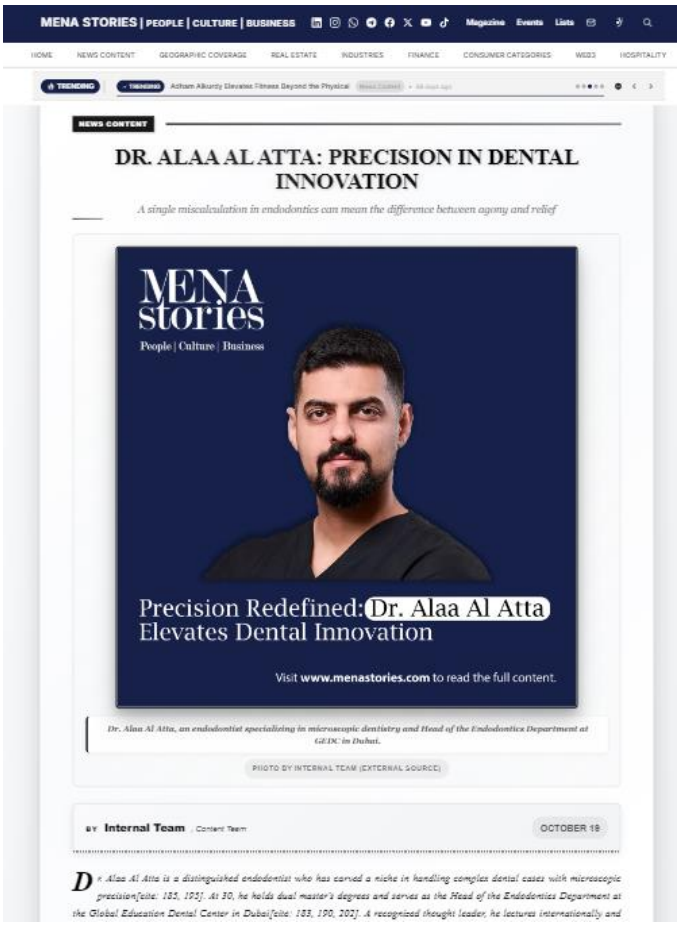
Admin Login



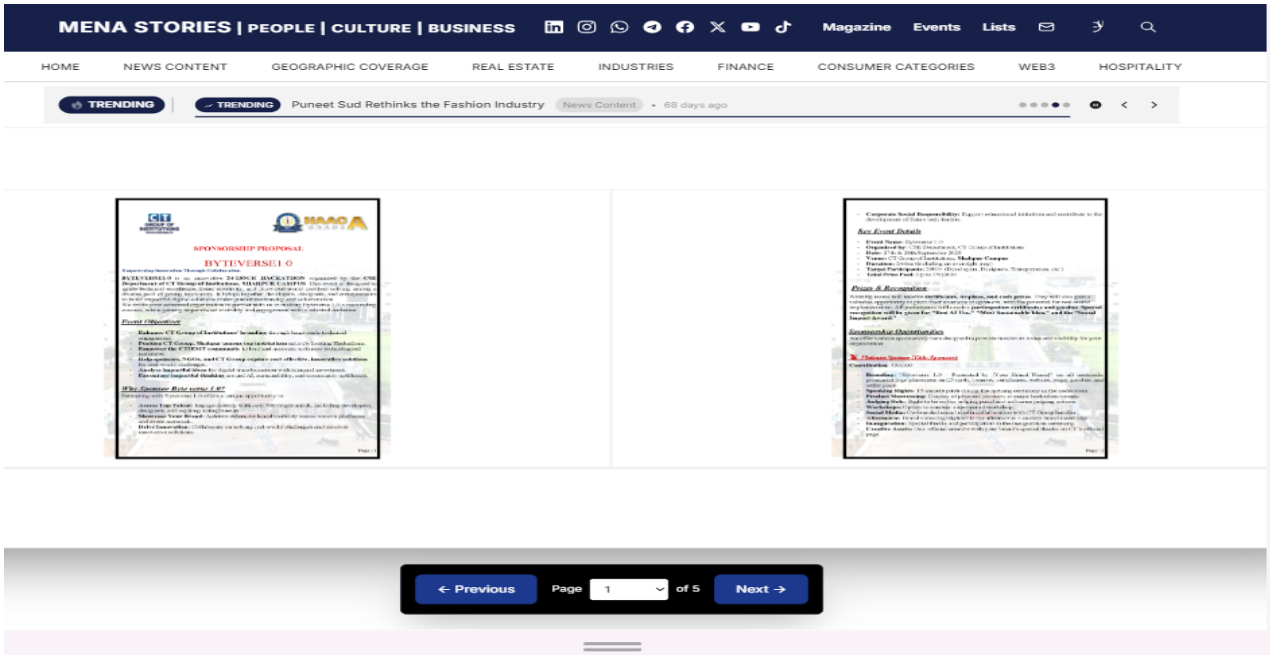
Homepage



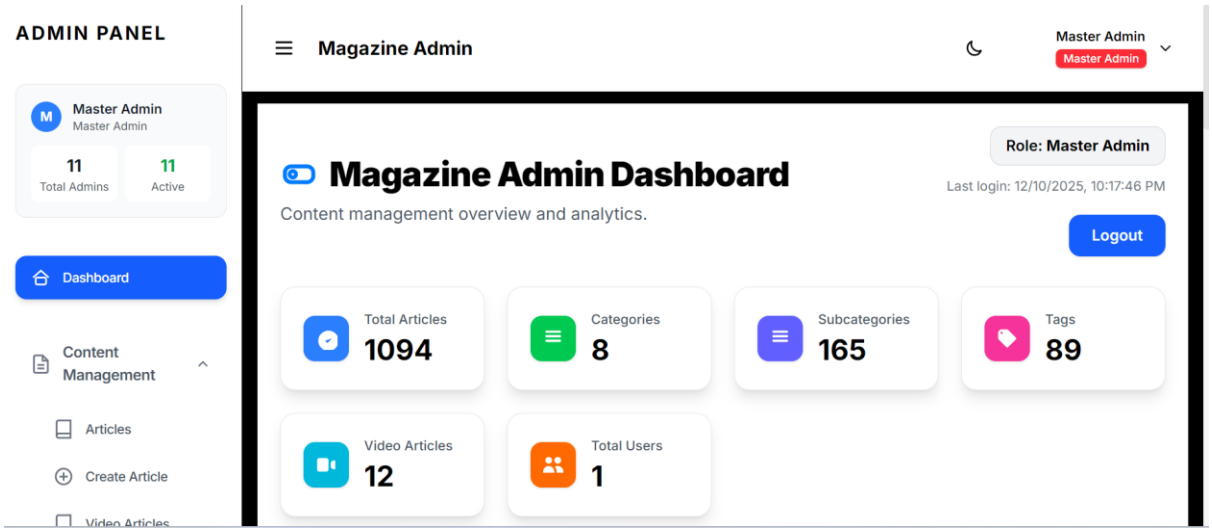
ArticlePage



FlipBook Page



Admin Dashboard



CHAPTER – 11

CONCLUSION

The MenaStories.me platform showcases the successful development of a modern, scalable, and secure digital publishing system built entirely with open-source technologies. By combining a robust React frontend, an optimized Node.js/Express backend, and a well-structured PostgreSQL database, the project delivers enterprise-grade performance while maintaining exceptional cost efficiency.

Through careful attention to security, performance optimization, and DevOps best practices, the platform achieves high reliability, low latency, and seamless deployment on VPS infrastructure. Its modular architecture ensures long-term maintainability and provides a strong foundation for future enhancements such as AI-driven features, multilingual support, and large-scale user growth.

Overall, this project highlights the effectiveness of full-stack development skills and modern engineering practices in creating a professional, production-ready digital publishing solution that can rival commercial platforms at a fraction of the cost.

CHAPTER-12

BIBLIOGRAPHY

12.1 Official Documentation

1. React Documentation (2024). React Official Documentation. Facebook Inc.
   - https://react.dev/
2. Node. js Documentation (2024). Node.js Official Documentation. OpenJS Foundation.
   - https://nodejs.org/docs/
3. Express.js Documentation (2024). Express.js Guide. OpenJS Foundation.
   - https://expressjs.com/
4. PostgreSQL Documentation (2024). PostgreSQL 14 Documentation. PostgreSQL Global Development Group.
   - https://www.postgresql.org/docs/14/
5. Sequelize Documentation (2024). Sequelize ORM Documentation.
   - https://sequelize.org/docs/v6/
6. Nginx Documentation (2024). Nginx Official Documentation. F5 Networks.
   - https://nginx.org/en/docs/
7. Vite Documentation (2024). Vite - Next Generation Frontend Tooling.
   - https://vitejs.dev/
8. Tailwind CSS Documentation (2024). Tailwind CSS Framework.
   - https://tailwindcss. com/docs

12.2 Books and Publications

9. Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
10. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps* (2nd ed.). O'Reilly Media.
11. Brown, E. (2019). *Web Development with Node and Express* (2nd ed.). O'Reilly Media.
12. Momjian, B. (2017). *PostgreSQL: Introduction and Concepts*. Addison-Wesley.
13. DeJonghe, D. (2019). *NGINX Cookbook*. O'Reilly Media.

12.3 Online Resources

14. MDN Web Docs (2024). Mozilla Developer Network - Web Development Resources.
- https://developer.mozilla.org/
15. Hostinger VPS Documentation (2024). VPS Hosting Guides and Tutorials.
- https://www.hostinger.com/tutorials/vps
16. DigitalOcean Community (2024). Server Administration Tutorials.
- https://www.digitalocean.com/community/tutorials
17. Cloudinary Documentation (2024). Media Management API Reference.
- https://cloudinary.com/documentation
18. Brevo (Sendinblue) API Documentation (2024). Email API Reference.
- https://developers.brevo.com/
19. Let's Encrypt Documentation (2024). ACME Protocol and Certbot.
- https://letsencrypt.org/docs/
20. PM2 Documentation (2024). Process Manager for Node.js.
- https://pm2.keymetrics.io/docs/

12.4 Research Papers and Articles

21. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.
22. JWT. io (2024). Introduction to JSON Web Tokens.
- https://jwt.io/introduction
23. OWASP Foundation (2024). OWASP Top Ten Web Application Security Risks.
- https://owasp.org/www-project-top-ten/

12.5 GitHub Repositories

24. Project Repository: MadhavArora1213/menastories. me
- https://github.com/MadhavArora1213/menastories.me